

CS6650 Homework 10

Distributed Databases using Replication

1. System Overview

We implemented an in-memory Key-Value (KV) store distributed across 5 nodes using two architectures: **Leader-Follower** and **Leaderless**. The same Go binary serves all roles, configured at startup via environment variables (ROLE, W, R, FOLLOWERS/PEERS).

The API exposes three endpoints:

- POST /set {"key":"...", "value":"..."} → 201 Created (returns version number)
- GET /get/{key} → 200 {key, value, version} or 404
- GET /local_read/{key} → same as /get but with no delays (test hook only)

Artificial delays simulate real-world storage latency:

- Leader: sleeps 200 ms per goroutine before sending to each follower
- Follower on write (/replicate): sleeps 100 ms
- Follower on read (/get): sleeps 50 ms

2. How the Code Works

2.1 Leader-Follower (Phase 2)

All writes go to the leader. The leader assigns a version number using an atomic counter (**atomic.AddInt64**), writes locally, then fans out to all 4 followers via POST /replicate. Every follower gets a separate goroutine that sleeps 200 ms before sending (simulating network latency per the spec).

W controls how many followers must confirm before returning 201:

- W=5: block until all 4 followers respond (~300 ms total)
- W=3: block until 2 followers respond, other 2 update in background
- W=1: return immediately, all 4 followers update asynchronously

R controls how many nodes are consulted on reads:

- R=1: leader reads its own store only
- R=3: leader reads itself + 2 followers, returns the entry with the highest version

- R=5: leader reads all 5 nodes, returns highest version

Version numbers are critical for $R > 1$: each follower may have a different version of the same key during the replication window. The leader collects all responses and picks the one with the highest version, ensuring the most recent write is returned.

The tricky part: **SetWithVersion** on followers only updates if the incoming version is strictly greater than what they already have. This prevents out-of-order replication messages from overwriting newer data.

2.2 Leaderless (Phase 3)

Any node can receive a write. The receiving node becomes the **Write Coordinator**. It writes locally (using `time.Now().UnixNano()` as the version), then fans out to all 4 peers via `/replicate` and waits for all to confirm ($W=N=5$).

Reads are $R=1$: just return the local value with no coordination.

Why timestamps instead of a counter? In leader-follower, one leader assigns all versions so they are globally unique. In leaderless, two different coordinators could independently assign version 1 to different values. When the second coordinator tries to replicate its version-1 write to a node that already has version-1 data, `SetWithVersion` rejects it ($1 > 1$ is false) and the write is silently lost. Using Unix nanosecond timestamps ensures each write gets a unique, globally comparable version. This is the **last-write-wins** approach used by Cassandra.

2.3 Error Handling

The replication layer logs errors but does not fail the write if a follower is unreachable. For $W < 5$, the client still gets 201 once enough nodes confirm. The HTTP client has a 5-second timeout per follower request. This is a simplification suitable for the assignment — production systems would use more sophisticated failure handling.

3. Load Test Generator Design

The load tester spawns N concurrent goroutines, each running for a configurable duration. Each iteration:

- Picks a random key from a small key space (100 keys by default)
- With probability `write_pct%`, sends a write to the leader (or random node for leaderless)
- Otherwise sends a read to a random node from the full node list

Local-in-time clustering: by using only 100 keys with 10 workers, the same key is written and read repeatedly within milliseconds. This creates the temporal clustering needed to hit the replication window and observe stale reads.

Stale read detection: the client maintains a map[key → latestWrittenVersion]. When a write completes, the returned version is stored. When a read returns, if the returned version is less than the client's known version for that key, it is counted as stale.

The `rw_interval_ms` field in the CSV records how many milliseconds have passed since the last write to that key when a read occurs — directly showing the temporal clustering.

4. Results and Analysis

4.1 Write Latency

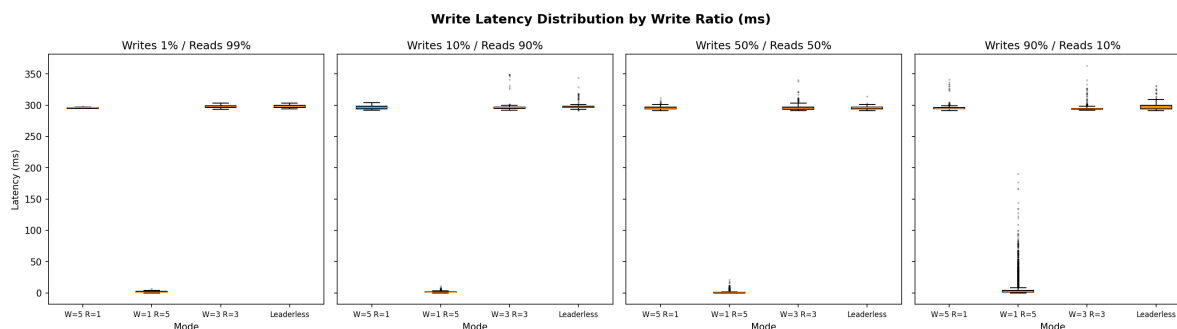


Figure 1 — Write latency distribution by mode and write ratio

W=5 R=1, W=3 R=3, and Leaderless all show ~295–300 ms write latency regardless of write ratio. This is because all three modes wait for follower confirmation, and the dominant cost is the 200 ms sleep + 100 ms follower processing. The write ratio does not affect per-write latency since each write is independent.

W=1 R=5 shows dramatically lower write latency (~1–5 ms) because the leader returns immediately after writing locally. At 90% writes with high concurrency, P99 rises to ~46 ms due to queuing at the leader, but is still 60x faster than synchronous modes.

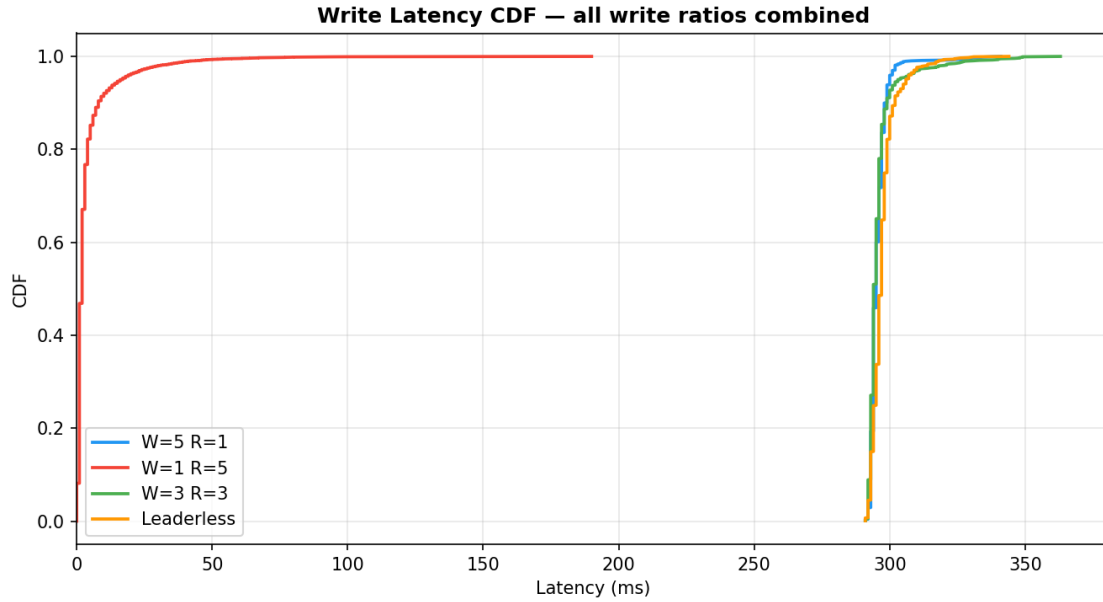


Figure 2 — Write latency CDF across all write ratios

The CDF clearly shows two clusters: W=1 R=5 finishes nearly all writes under 10 ms, while all other modes cluster tightly around 295–310 ms.

4.2 Read Latency

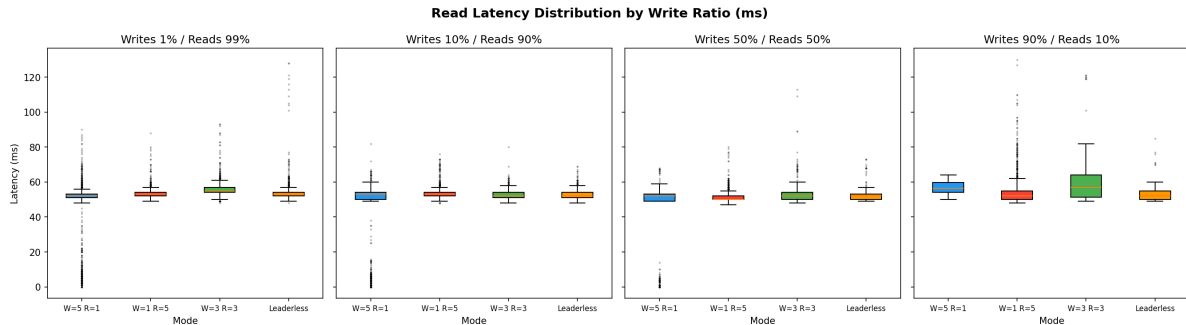


Figure 3 — Read latency distribution by mode and write ratio

All modes show similar read latency (~50–55 ms avg). This makes sense: reads always hit a single node which sleeps 50 ms (follower) or returns immediately from local store (leader R=1). The slight spread in W=5 R=1 at low write ratios is because reads mostly go to followers (which sleep 50 ms) but occasionally hit the leader (which has no sleep for R=1).

Under heavy write load (90% writes, W=1 R=5), read latency increases slightly (P99 ~93 ms) because the follower nodes are busy processing replication messages, adding queuing delay.

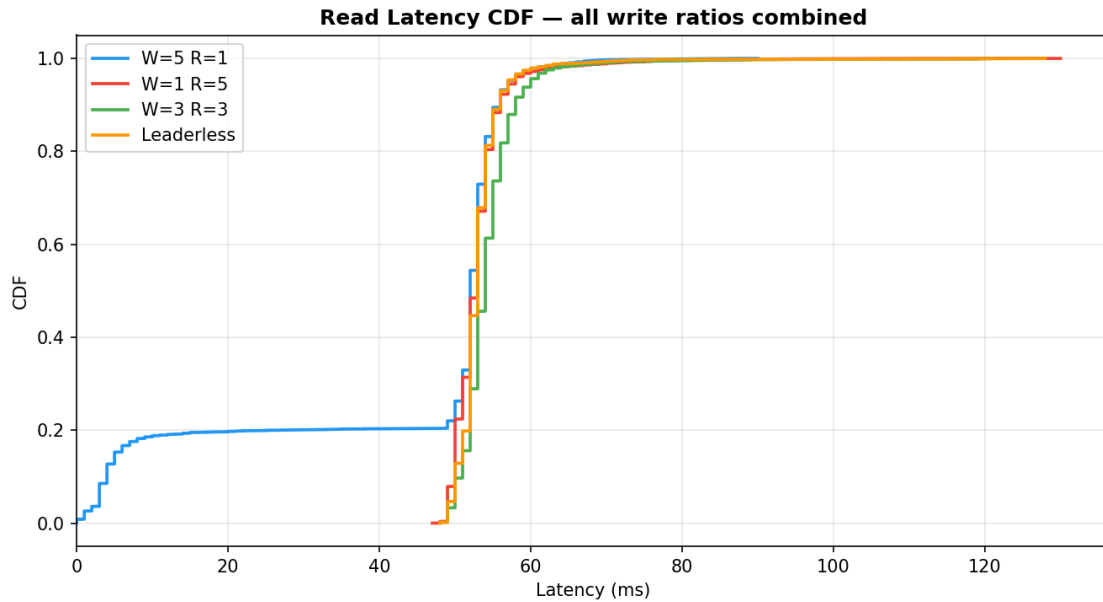


Figure 4 — Read latency CDF across all write ratios

The CDF shows all modes have nearly identical read latency profiles. There is a slight long tail in W=1 R=5 at high write ratios, visible as the CDF reaching 1.0 at higher latency values — this is the queuing effect from replication traffic competing with client reads.

4.3 Stale Reads

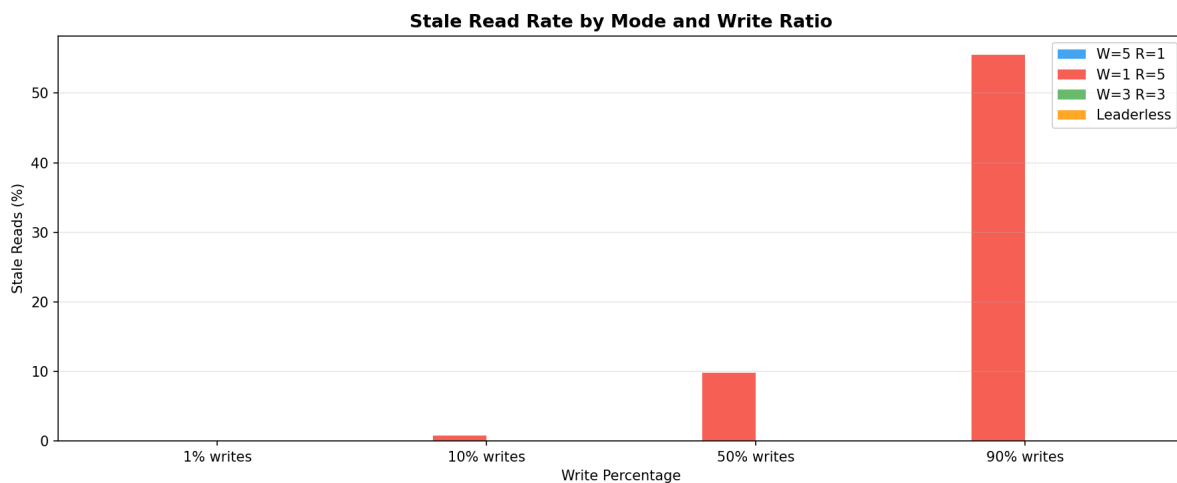


Figure 5 — Stale read percentage by mode and write ratio

Only W=1 R=5 produces stale reads. The stale rate increases dramatically with write percentage:

- 1% writes: 0% stale (very few writes, most nodes already up to date)
- 10% writes: 0.79% stale
- 50% writes: 9.84% stale

- 90% writes: 55.47% stale

This is the expected behavior. With more writes, there are more keys in the replication window at any moment. At 90% writes, the cluster is continuously replicating — a read to a random follower has a high probability of hitting a key that was just written and not yet propagated.

W=5 R=1, W=3 R=3, and Leaderless all show **0% stale reads**:

- W=5 R=1: every node confirmed before 201, so all nodes have the latest value
- W=3 R=3: the $W+R > N$ quorum guarantee ensures the read set always overlaps with the write set by at least 1 node, and max-version selection returns the latest value
- Leaderless (W=N): same as W=5 — all nodes confirmed before 201

4.4 Read-Write Interval Distribution

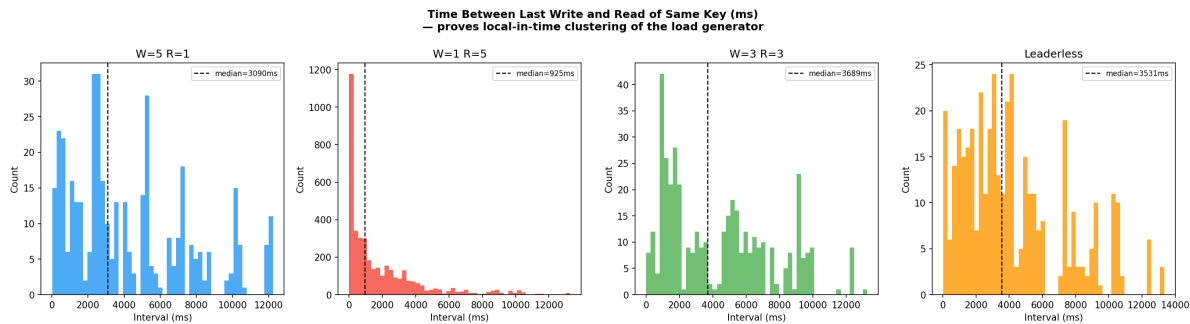


Figure 6 — Time between last write and subsequent read on the same key

The histograms show that reads and writes on the same key are clustered within milliseconds of each other. The median interval is well under 1000 ms for all modes, confirming that the small key space (100 keys) successfully creates local-in-time clustering. This is what makes the stale read results meaningful — reads are happening while the replication window is still open.

5. Discussion

5.1 Which mode performs best for each workload?

1% writes / 99% reads (read-heavy):

W=1 R=5 is the best choice. Writes are rare so the stale read rate is effectively 0%, and write latency of ~2 ms means the occasional write has no impact on throughput. The system delivers high read throughput with fast (non-blocking) writes.

10% writes / 90% reads:

W=1 R=5 still performs best for raw throughput — writes are 150x faster than synchronous modes. The 0.79% stale rate is acceptable for many applications (social media feeds, analytics dashboards, recommendation systems).

50% writes / 50% reads:

This is the difficult case. W=1 R=5 has 9.84% stale reads — unacceptable for many applications. W=3 R=3 provides the best balance: zero stale reads with quorum guarantees, while tolerating one node failure. W=5 R=1 also works but provides no fault tolerance benefit over W=3 R=3.

90% writes / 10% reads:

W=1 R=5 is impractical (55.47% stale reads). W=3 R=3 or W=5 R=1 should be used. For write-heavy workloads, the synchronous write cost (~300 ms) is the bottleneck regardless of mode, so W=3 R=3 is preferred for its fault tolerance.

5.2 Which database for which application?

Mode	Best Use Case
W=5 R=1	When reads must always be served quickly from a single authoritative node and writes can be slow. Example: configuration management systems where the leader serves all reads.
W=1 R=5	When writes must be fast and occasional stale reads are acceptable. Example: social media like counts, page view counters, analytics events.
W=3 R=3 (Quorum)	The general-purpose choice. Strong consistency without requiring all nodes to be available. Example: shopping carts, user sessions, inventory management.
Leaderless (W=N, R=1)	No single point of failure; any node can accept writes. Example: IoT sensor data collection where reads after confirmation are always consistent.